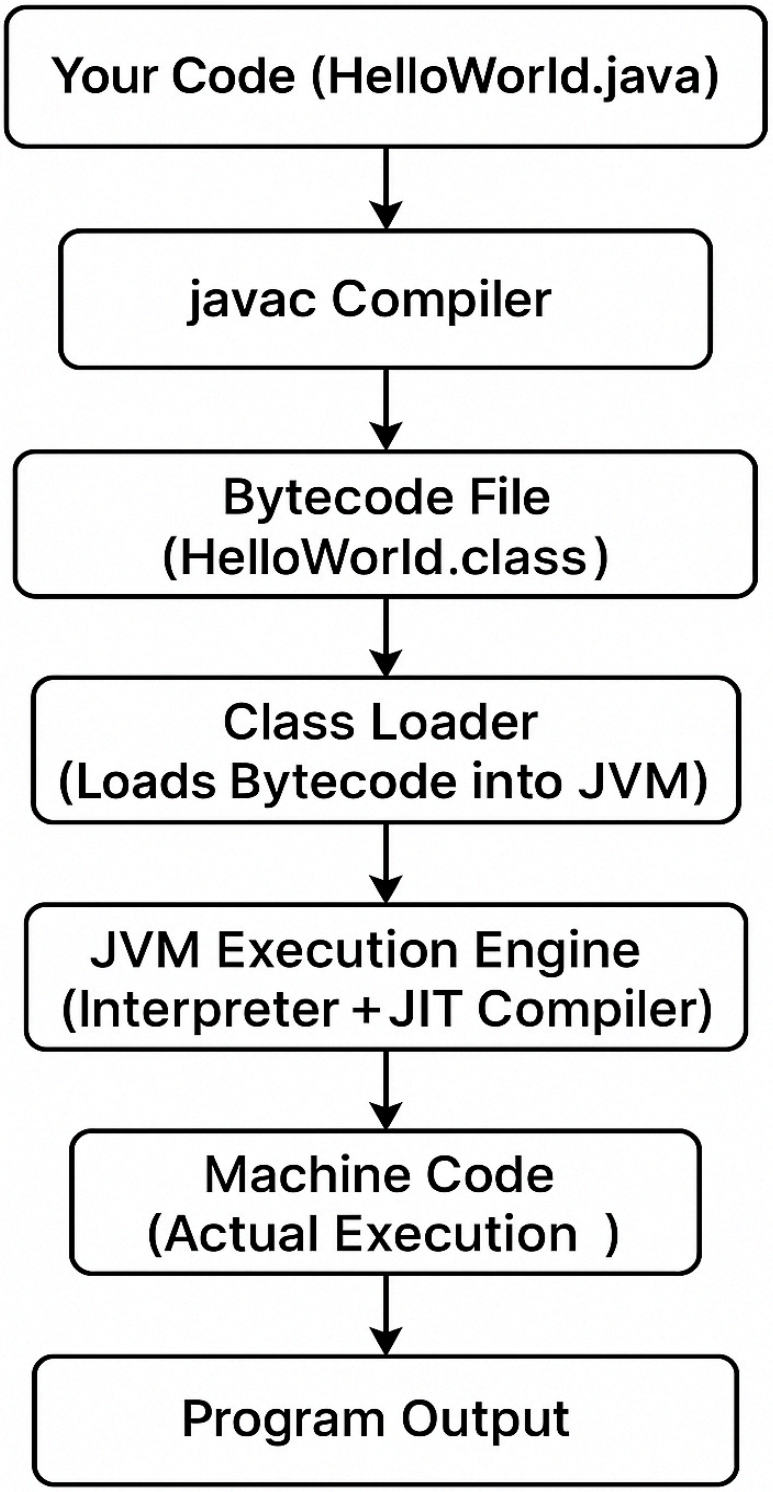
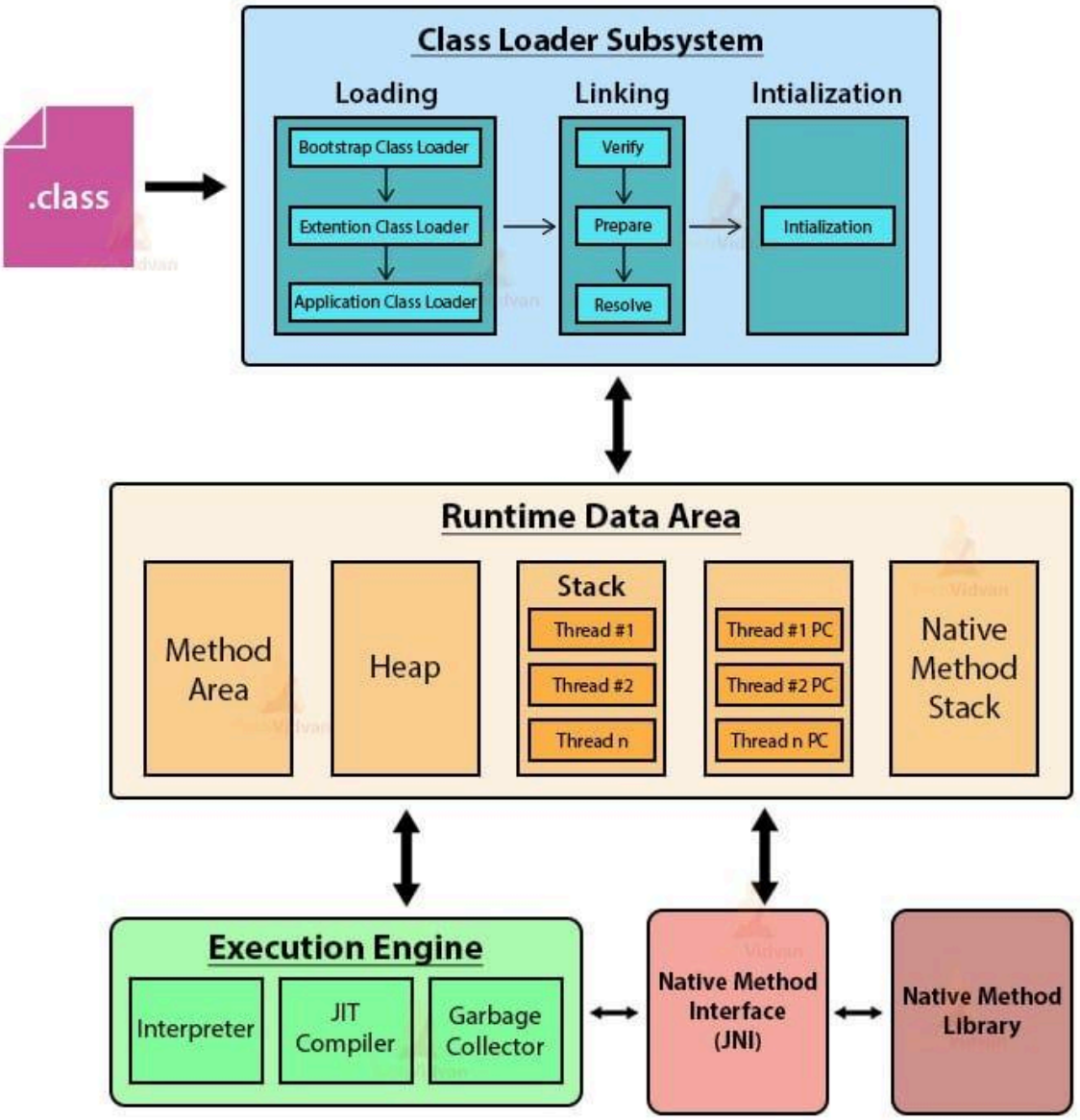
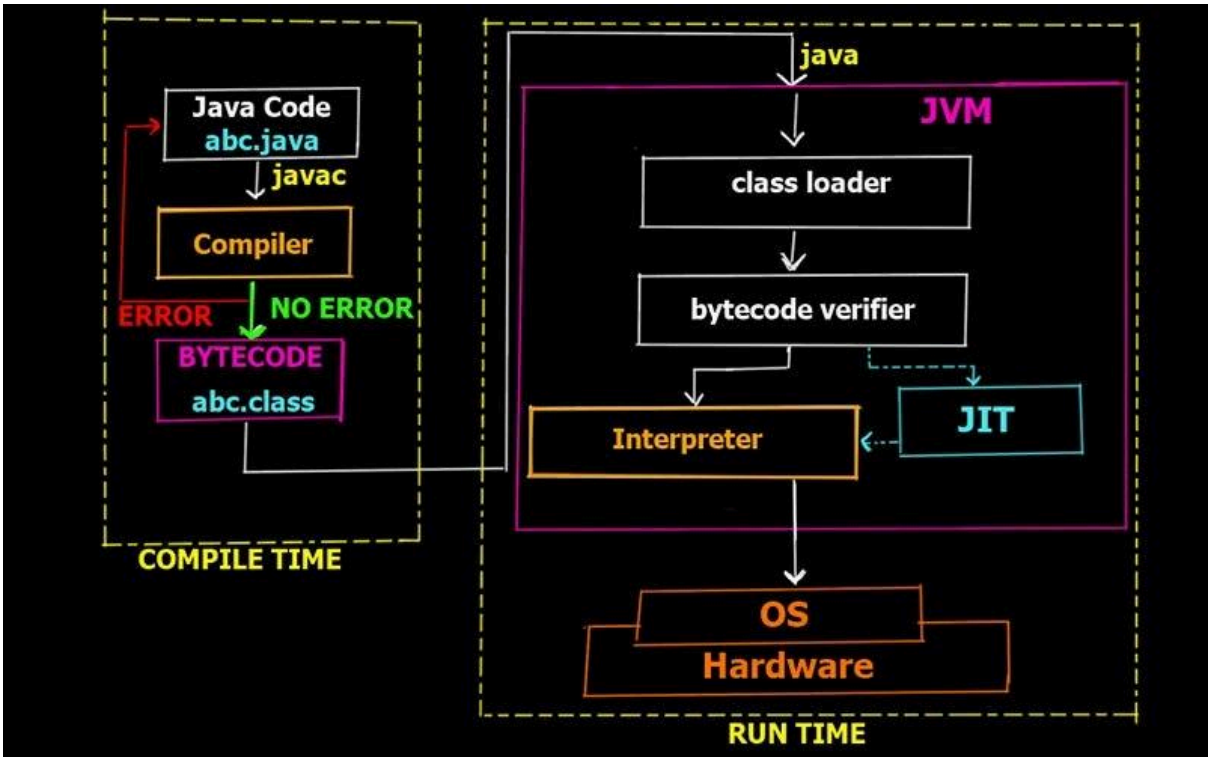
☕ Compilation vs Execution in Java



JVM Model


.class
Class Loader Subsystem
Loading
Linking
Intialization
Bootstrap Class Loader
Extention Class Loader
Application Class Loader
Verify
Prepare
Resolve
Intialization
Runtime Data Area
Method Area
Heap
Stack
Thread #1
Thread #2
Thread n
Thread #1 PC
Thread #2 PC
Thread n PC
Native Method Stack
Execution Engine
Interpreter
JIT Compiler
Garbage Collector
Native Method Interface (JNI)
Native Method Library



Understanding this is very important for interviews and fundamentals.

🧱 Compilation (Compile Time)

🔹 What is Compilation?

Compilation is the process of converting **Java source code (.java)** into **bytecode (.class)** using the Java compiler.

🔹 Tool used:

`javac` (Java compiler)

🔹 Command:

javac Hello.java

🔹 Output:

- Generates `Hello.class` file
- Contains **bytecode**, not machine code

▶️ Execution (Run Time)

🔹 What is Execution?

Execution is the process of running the **bytecode** using JVM.

🔹 Tool used:

`java` command + JVM

🔹 Command:

java Hello

🔹 Output:

Program result printed on console.

🔄 Flow of Java Program

Java Code (.java)
↓
Compilation (javac)
↓
Bytecode (.class)
↓
Execution (JVM)
↓
Machine Code → Output

🆚 Compilation vs Execution (Difference)

Feature	Compilation	Execution
Stage	Before running program	Running the program
Input	.java file	.class file
Output	Bytecode	Program output

Tool	javac	java + JVM
Error type	Syntax errors	Runtime errors

🎤 Interview One-Line Answer

> Compilation converts Java source code into bytecode using `javac`, while execution runs that bytecode using the JVM to produce output.

💡 Example

File: `Hello.java`

Compile:

javac Hello.java

Run:

java Hello

⚠️ Important Interview Point

- Compilation happens **once**
- Execution happens **every time you run the program**